AI Feature Architecture + Cost Analysis

A compact working reference for evaluating an AI feature from product problem through architecture, cost, and ongoing optimization.

> **CORE RULE: Start with the product problem, not the model. Choose the least-complex system that solves the problem well, then cost that architecture.**

A. Decide What to Build

1. Define the problem

What is difficult today? What outcome should improve? Is the job retrieval, classification, extraction, synthesis, recommendation, reasoning, or action? Could ordinary software solve it?
Rewrite: “We want AI for X” -> “Users need to ____. What is the best way to enable that?”

2. Set the authority boundary

Complete: **“AI may ____, but it may not ____.”** Define what AI can read, infer, suggest, change, or write back; what requires approval; and what remains authoritative.

3. Map the information

Separate current state, goals, decisions, open questions, evidence, pending information, and history. **Retrieved does not automatically mean true.** Decide which information types outrank others.

4. Find the lowest sufficient capability

Move up only as needed: **conventional search -> better ranking/filters -> semantic search -> semantic + structured results -> LLM synthesis -> agents.**
Ask: *What useful capability does this add that the previous level cannot?*

5. Decide whether generation adds value

Good fits: synthesis, explaining change, comparison, briefing, prioritization, and grounded recommendations. Usually poor fits: known-field lookup, filtering, basic retrieval, or converting clear structured data into prose.

B. Design the AI Path

6. Design retrieval and context

Combine semantic relevance with metadata, authority, recency, and user intent. Do not send every retrieved record to the LLM. Aim for **minimum sufficient context**.

7. Design trustworthy output

Keep authoritative facts, unknowns, pending evidence, history, and AI suggestions distinguishable. Make provenance inspectable. Ask whether a user could mistake AI reasoning for authoritative information.

8. Define failure modes

Separate **critical failures** from softer quality misses. Example: promoting unreviewed evidence into fact. Overall accuracy alone is not enough.

9. Evaluate models

Test representative tasks. Compare success, critical failures, groundedness, completeness, latency, tokens, and cost. Choose the **least-expensive model that reliably clears the quality bar.**

C. Model the Cost

Only price the feature after you know what the system does and how often it will do it.

Measure	How to estimate it
Monthly uses	active users x uses/user/day x active days
Generation calls	monthly uses x % requiring synthesis
Input tokens	system instructions + query + retrieved context
Output tokens	generated response
Generation cost	(input / 1M x input price) + (output / 1M x output price)
Other costs	embeddings, vector search/storage, DB/hosting, caching, retries, monitoring/evals, maintenance

D. Stress-Test the Assumptions

Build **low / base / high** scenarios. Vary the assumptions most likely to change. The high case should be plausible heavy usage, not a theoretical maximum.

Assumption	Low	Base	High
Users			
Uses/user/day			
% requiring generation			
Input tokens/call			
Output tokens/call			
Monthly AI cost			

Run sensitivity analysis

Change users, usage frequency, generation-routing rate, context size, output size, model, and retries. Ask: **Which assumptions actually move the bill?** Those become monitoring priorities.

E. Optimize and Operate

Optimize in this order: avoid unnecessary model calls -> improve retrieval/ranking -> remove unnecessary context -> control excessive output -> cache where useful -> test cheaper models -> consider limits or pricing.

Once live: monitor generation-routing rate, tokens/synthesis, cost/synthesis/customer, success rate, critical failures, retries/reformulations, and AI cost as % of revenue. Replace assumptions with observed behavior. Investigate expensive users before throttling them; heavy usage may indicate inefficiency or high value.

F. Final Check

Before recommending the architecture:

1. What problem are we actually solving?

2. Why does this need AI?

3. What is AI allowed to decide or change?

4. What is the least-complex capability that solves it?

5. What information does the model actually need?

6. How can it fail, and which failures matter most?

7. What is the cheapest model that clears our quality bar?

8. Which usage assumptions drive cost?

9. What happens if those assumptions are wrong?

10. What will we measure - and what evidence would make us change the architecture?